Project 2 – Acme Analytics Linux Platform

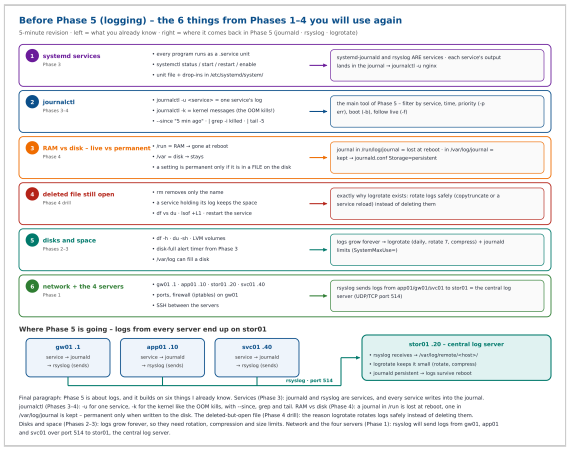
Day 14 · Phase 5 starts: how Linux runs a program and reads a file

Muhammad Bilal Javed · 6–7 November 2026 · study copy

Phase	5 – Logging (journald, rsyslog, logrotate) · Step 5.1: CPU, memory or disk – where is the bottleneck?
Hosts worked on	app01
Status	Step 5.1 🟡 – foundations ✅ (user/kernel space, system calls, process cards, file tables, inodes, map cache, page cache, CPU states, vmstat columns) · warm-up on app01 ✅ · experiments A, B, C ⏳

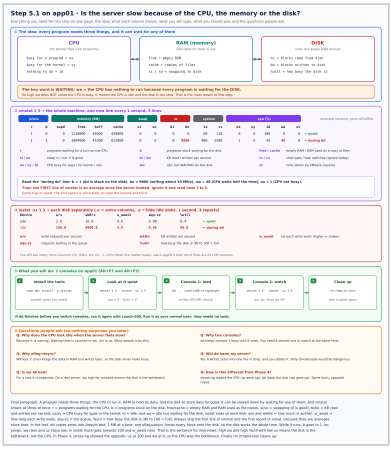
How to read this report: every section starts with the picture, then the commands with the real output from app01 under them, then a plain-language explanation. Each picture ends with a *final paragraph* that contains all corrections. New terms are in the glossary.

0. Before Phase 5 – quick revision



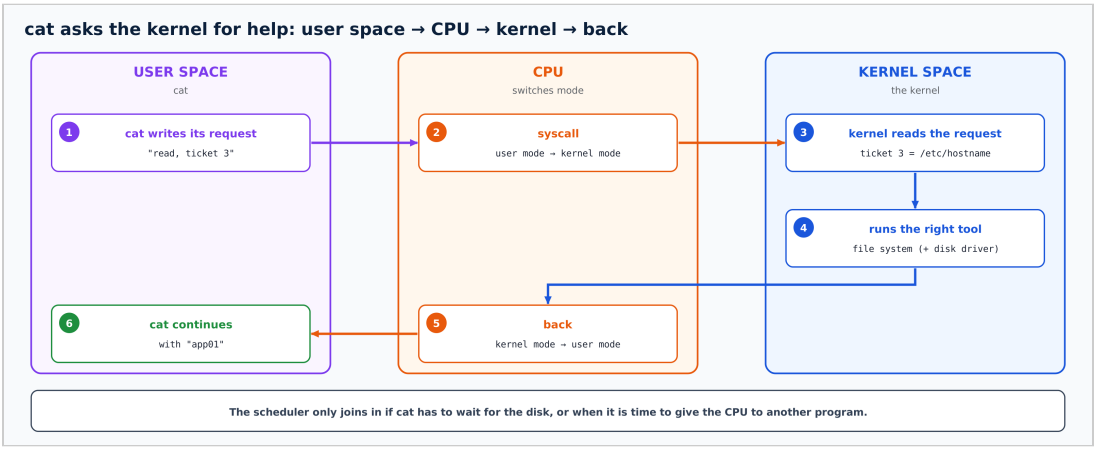
Before Phase 5 – revision

Phase 5 is about logs. Before collecting logs, Step 5.1 answers a simpler question first: **when a server is slow, which part is waiting – the CPU, the memory or the disk?** To read the answer from `vmstat` and `iostat`, you need to know what really happens inside Linux when a program runs and reads a file. That is what this day covers.



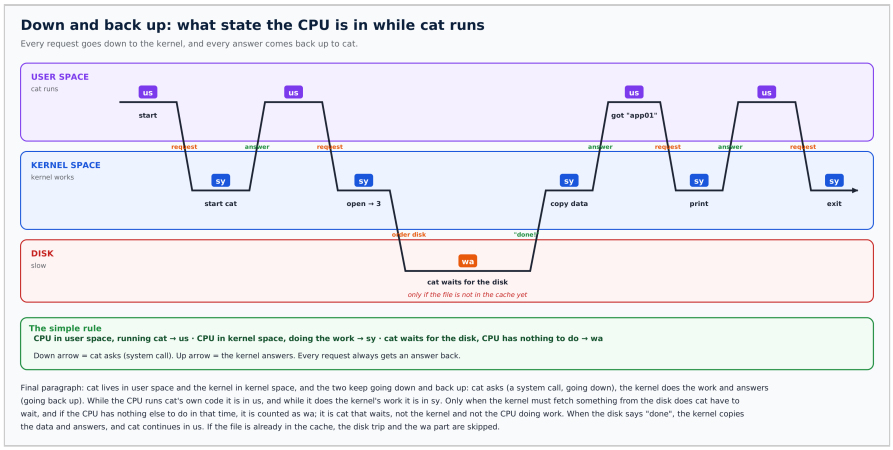
Step 5.1 on one page

1. User space, kernel space and the system call



user space → CPU → kernel → back

- **User space** is where programs live, each in its own walled room in RAM (its code and its data). The CPU runs them in **user mode** – counted as `us`.
- **Kernel space** is where the kernel lives. Only the kernel may touch the disk, the network or other programs. The CPU runs it in **kernel mode** – counted as `sy`.
- A program can only ask the kernel for help through a **system call** (open, read, write, fork, exec, exit). It writes a request number and the details into the CPU's registers and runs the `syscall` instruction; the CPU switches to kernel mode, the kernel does the job and the CPU switches back.



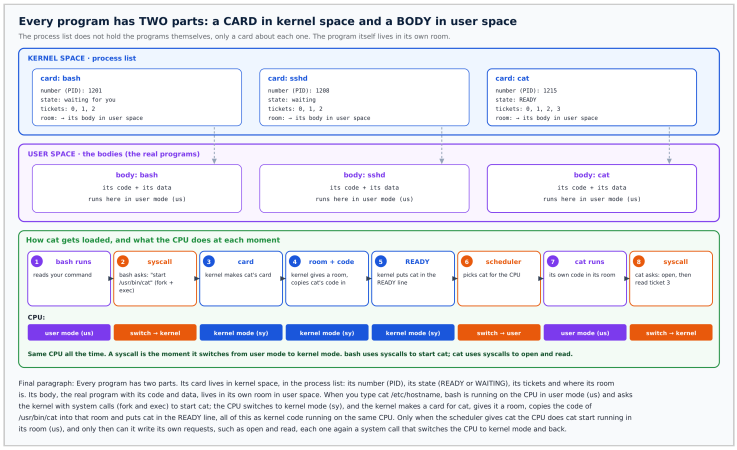
Down and back up – us, sy, wa

CPU time	Meaning
`us`	running a program's own code (user mode)
`sy`	running kernel code for it (kernel mode)
`wa`	idle, because a program is waiting for the disk
`id`	idle, nothing to do

Waiting is **never** counted as `us`.

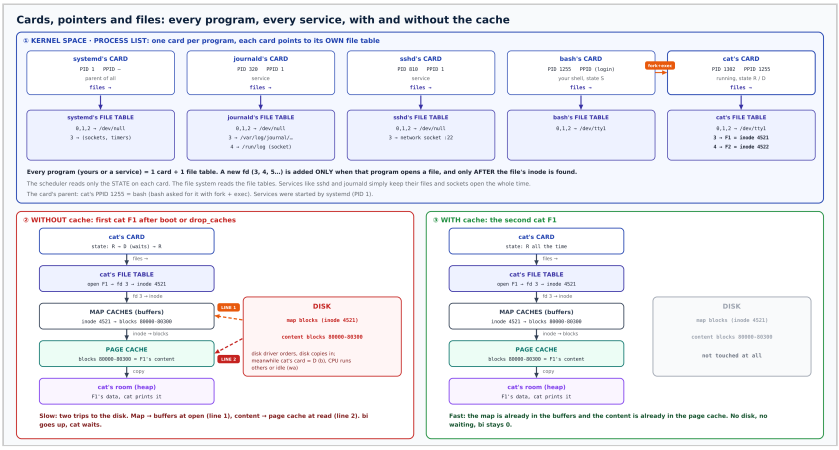
---

2. Every program = a card in kernel space + a body in user space



Card and body

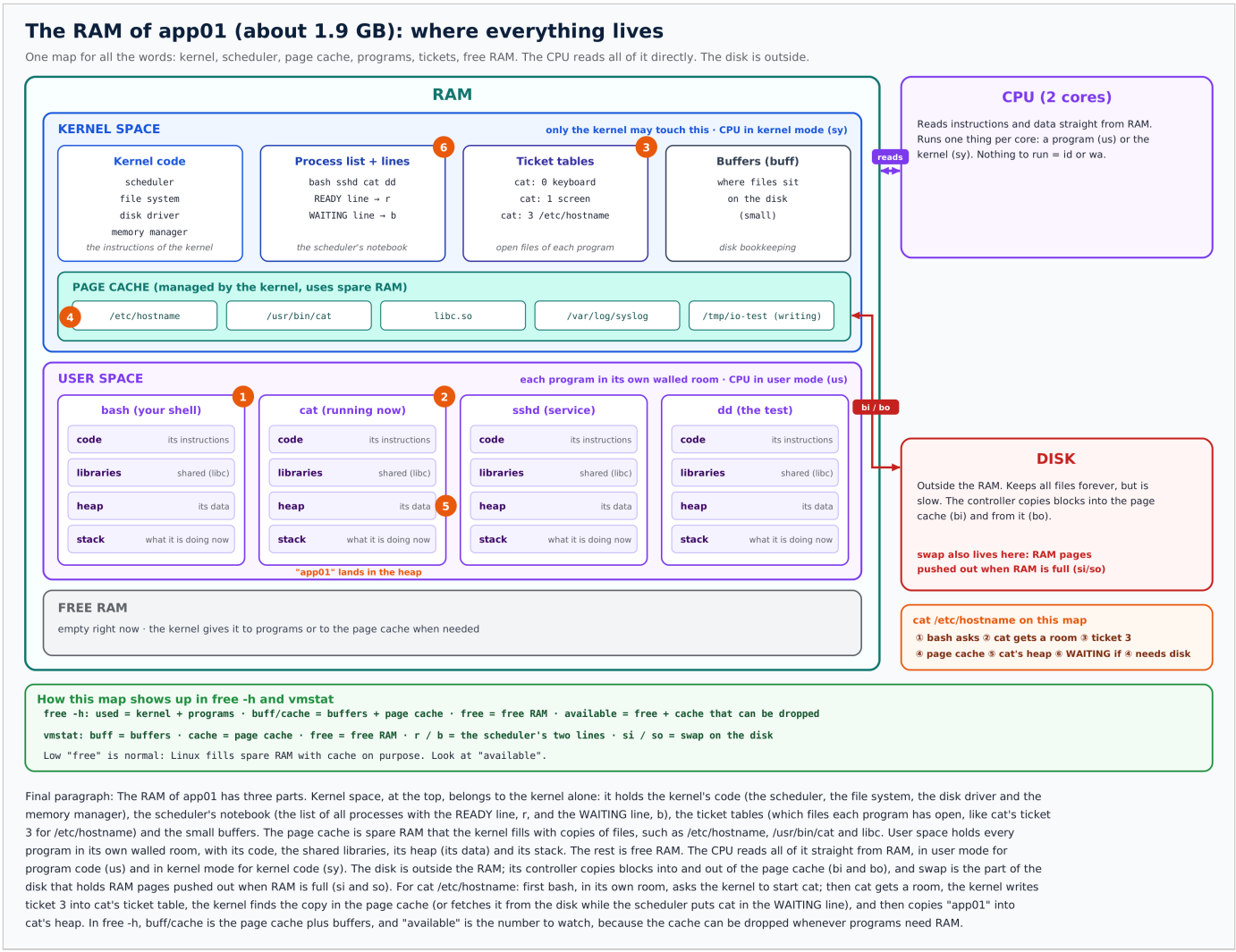
- When you type `cat F1`, **bash** asks the kernel with `fork` + `exec`. `fork` makes a **new card** (a new PID, parent = bash); `exec` puts cat's code into its new room. bash keeps its own card and waits.
- The card (real name: `task_struct`) holds the **PID**, the **parent PID**, the **state** and a pointer to the program's **file table** (fds 0, 1, 2 = keyboard, screen, errors).
- States: **R** = can run · **S** = sleeping (e.g. waiting for typing) · **D** = waiting for the disk.
- Every program and every service (systemd, journald, sshd, bash, cat) has its own card and its own file table.



Cards, pointers and files – with and without the cache

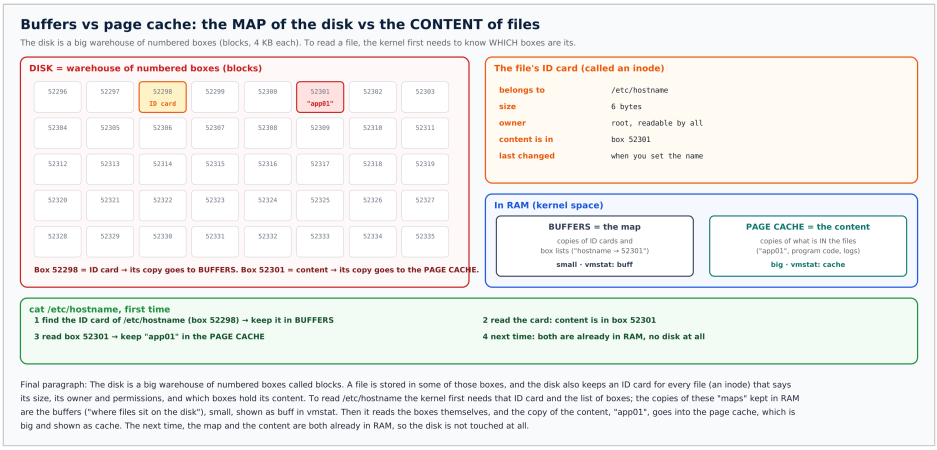
---

3. Where everything lives in RAM



Final paragraph: The RAM of app01 has three parts. Kernel space, at the top, belongs to the kernel alone: it holds the kernel's code (the scheduler, the file system, the disk driver and the memory manager), the scheduler's notebook (the list of all processes with the READY line, r, and the WAITING line, b), the ticket tables (which files each program has open, like cat's ticket 3 for /etc/hostname) and the small buffers. The page cache is spare RAM that the kernel fills with copies of files, such as /etc/hostname, /usr/bin/cat and libc. User space holds every program in its own walled room, with its code, the shared libraries, its heap (its data) and its stack. The rest is free RAM. The CPU reads all of it straight from RAM, in user mode for program code (us) and in kernel mode for kernel code (sy). The disk is outside the RAM; its controller copies blocks into and out of the page cache (bi and bo), and swap is the part of the disk that holds RAM pages pushed out when RAM is full (si and so). For cat /etc/hostname: first bash, in its own room, asks the kernel to start cat; then cat gets a room, the kernel writes ticket 3 into cat's ticket table, the kernel finds the copy in the page cache (or fetches it from the disk while the scheduler puts cat in the WAITING line), and then copies "app01" into cat's heap. In free -h, buff/cache is the page cache plus buffers, and "available" is the number to watch, because the cache can be dropped whenever programs need RAM.

The RAM of app01

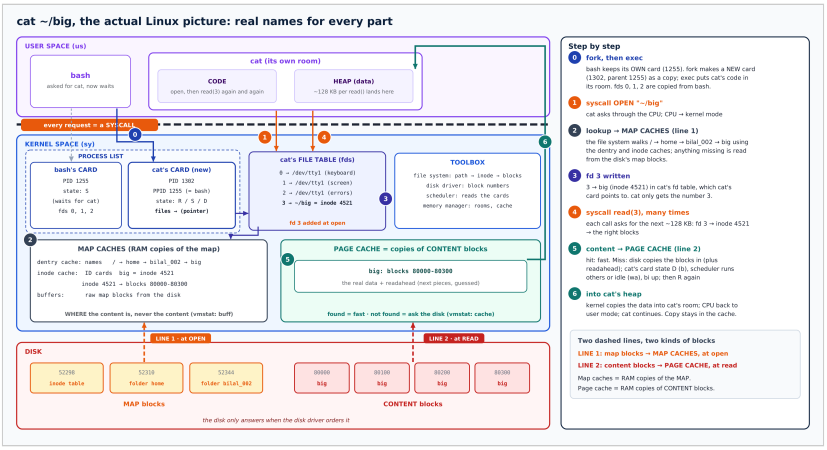


Buffers vs page cache

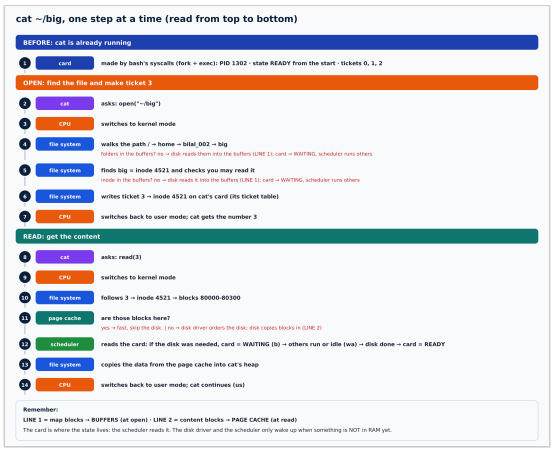
- **Map cache** (buffers, inode cache, dentry cache): RAM copies of the *map* – folder entries (name → inode number) and inodes (owner, size, **list of content blocks**). It holds **where** the content is, never the content.
- **Page cache**: RAM copies of file **content**.
- The disk keeps everything for good; the **disk driver** only fetches what RAM is missing.

---

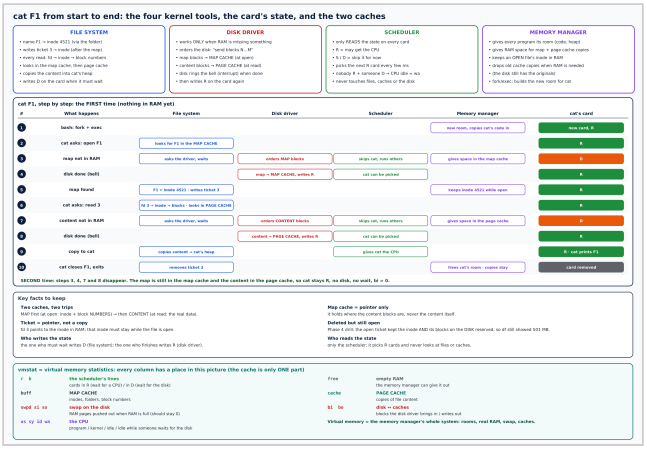
4. `cat F1` from start to end – who does what



Open and read – the actual Linux picture



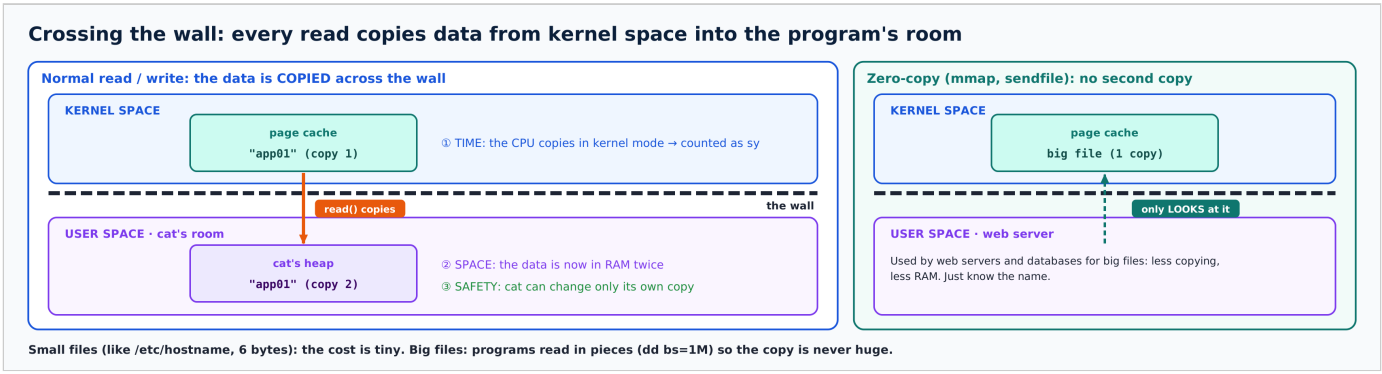
cat F1 step by step



The four kernel tools, the card's state and vmstat

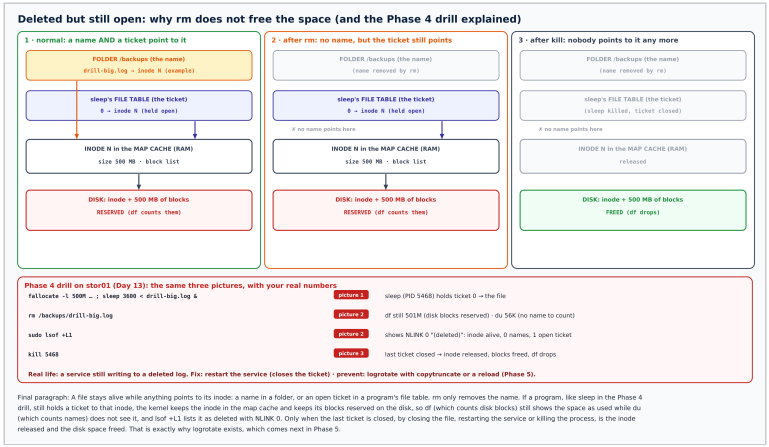
1. **open F1** – the file system looks up the map: folder → `F1 = inode 4521` → inode with the block list. Missing map blocks are fetched from the disk into the map cache (**line 1**).
2. Only **after** the inode is found is the ticket written: `3 → inode 4521` in cat's file table.
3. **read(3)** – fd 3 → inode → block numbers → page cache. Missing content blocks are fetched from the disk into the page cache (**line 2**).
4. While the disk works, the file system writes **D** on cat's card; the scheduler (which only reads the states) runs other programs or the CPU is idle (`wa`). When the disk rings its bell, the disk driver writes **R** again.
5. The content is copied from the page cache into cat's heap and cat continues. The second time, everything is already in RAM: no disk, no wait.

Every read and write crosses the wall as a **copy** – a little `sy` time and the data sits in RAM twice, but each program can only change its own copy:



Crossing the wall

5. Link to Phase 4: deleted but still open



Deleted but still open – explained

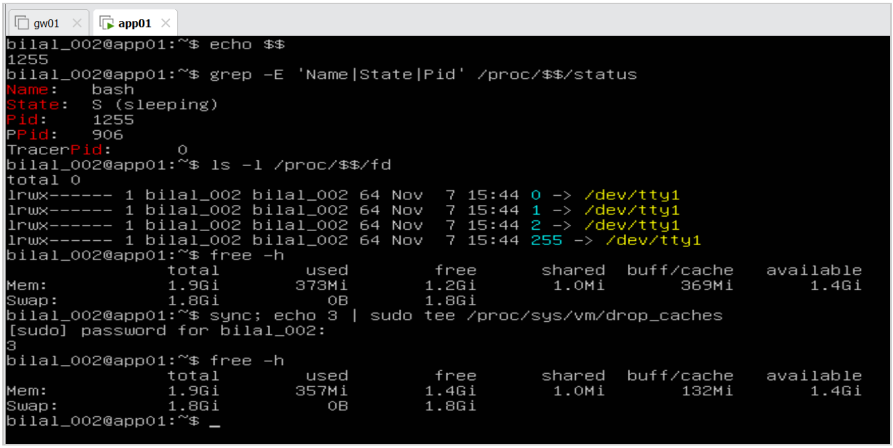
This explains the **first incident drill of Phase 4** (Day 13, section 2):

Drill step on stor01	What happened inside
`sleep 3600 < /backups/drill-big.log &`	sleep's file table holds a ticket (fd 0) to the file's inode
`rm /backups/drill-big.log`	only the **name** is removed; the ticket still points to the inode
`df` 501M · `du` 56K	the inode stays in the map cache and its blocks stay **reserved on the disk**; `du` counts names, `df` counts disk blocks
`lsof +L1` → NLINK 0 "(deleted)"	the inode is alive with 0 names and 1 open ticket
`kill 5468`	the last ticket is closed → inode released, blocks freed, `df` drops

In real life this is a service still writing to a deleted log – and the reason **logrotate** exists (later in Phase 5).

6. Warm-up on app01 – seeing it live

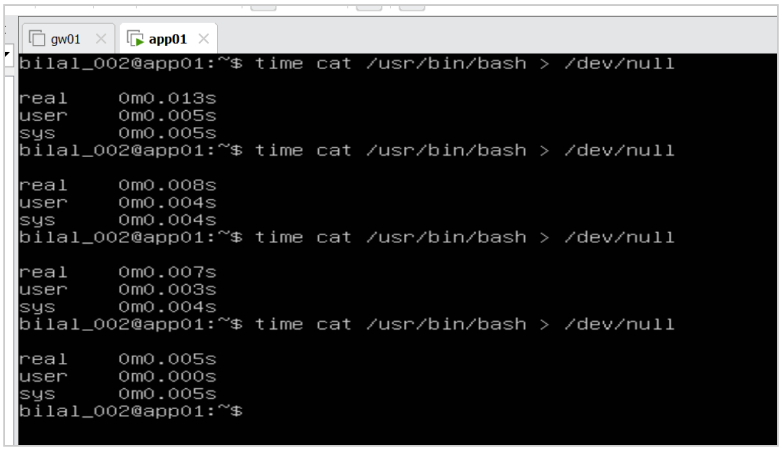
```
echo $$
grep -E 'Name|State|Pid' /proc/$$/status
ls -l /proc/$$/fd
free -h
sync; echo 3 | sudo tee /proc/sys/vm/drop_caches
free -h
```



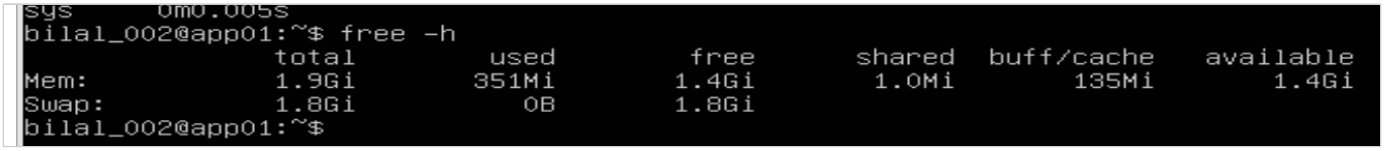
bash's card, tickets, cache emptied

bash = PID 1255, state S (waiting for typing), parent 906 (the login program); tickets 0, 1, 2 point to the console `/dev/tty1` (255 is bash's own extra ticket). Dropping the caches took buff/cache from **369 Mi to 132 Mi** – the rest is in use by running programs.

```
time cat /usr/bin/bash > /dev/null     # 4 times
free -h
```



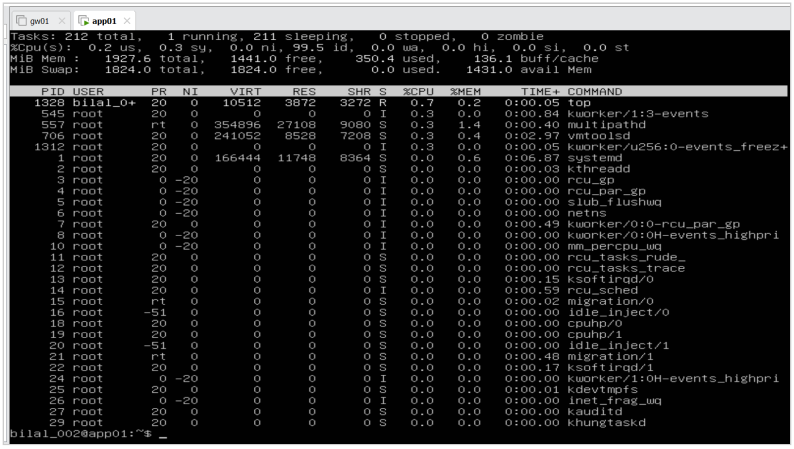
time cat – first from disk, then from cache



buff/cache grew again

First read **13 ms** (from the disk), then 8, 7, **5 ms** (from the page cache); buff/cache grew from 132 to 135 Mi. The gap is small because the file is small and parts of bash were still in RAM.

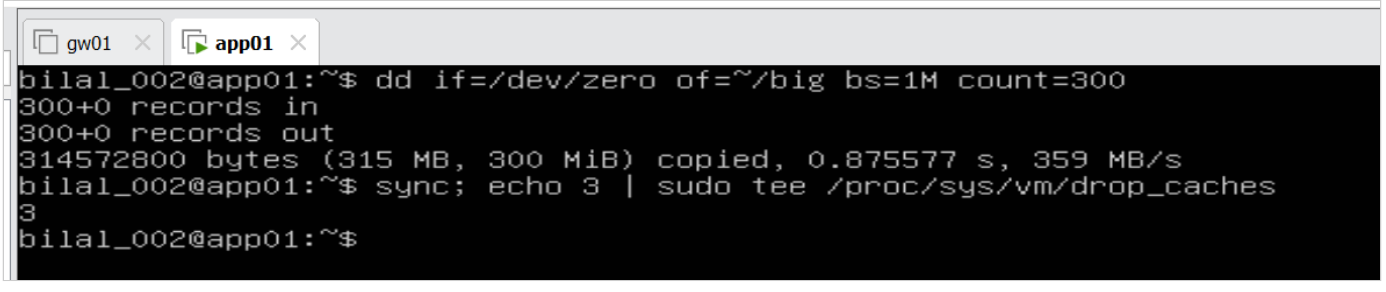
```
top
```



top – 212 cards, 1 running

212 tasks: 1 running, 211 sleeping – 212 cards in the process list (systemd PID 1, vmtoolsd, multipathd, many kernel `kworker` helpers); CPU **99.5 % id, 0.0 wa**: nobody ready, nobody waiting for the disk.

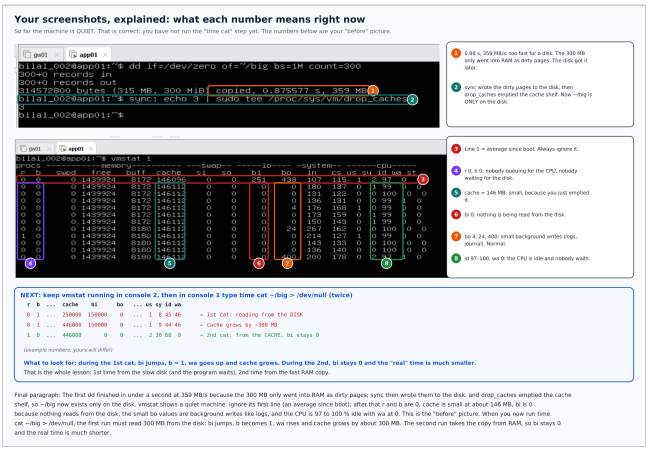
```
dd if=/dev/zero of=~/big bs=1M count=300
sync; echo 3 | sudo tee /proc/sys/vm/drop_caches
vmstat 1          # second console
```



dd 300 MB in 0.88 s

```
bilal_002@app01:~$ vmstat 1
procs -----------memory---------- ---swap-- -----io---- -system-- ------cpu-----
 r  b   swpd   free   buff  cache   si   so    bi    bo   in   cs us sy id wa st
 0  0      0 1439924   8172 146096    0    0   251   438  107  115  1  2 97  0  0
 0  0      0 1439924   8172 146112    0    0     0     0  180  137  0  1 99  0  0
 0  0      0 1439924   8172 146112    0    0     0     0  131  122  0  0 100  0  0
 0  0      0 1439924   8172 146112    0    0     0     0  136  131  0  0 99  1  0
 0  0      0 1439924   8172 146112    0    0     0     4  176  168  1  0 99  0  0
 0  0      0 1439924   8172 146112    0    0     0     0  173  159  0  1 99  0  0
 0  0      0 1439924   8172 146112    0    0     0     0  150  143  0  1 99  0  0
 0  0      0 1439924   8180 146112    0    0     0    24  267  162  0  0 100  0  0
 1  0      0 1439924   8180 146112    0    0     0     0  214  127  1  0 99  0  0
 0  0      0 1439924   8180 146112    0    0     0     0  143  133  0  0 100  0  0
 0  0      0 1439924   8180 146112    0    0     0     0  136  140  0  0 100  0  0
 0  0      0 1439924   8180 146112    0    0     0   400  200  178  0  2 97  1  0
```

vmstat – the quiet “before” picture



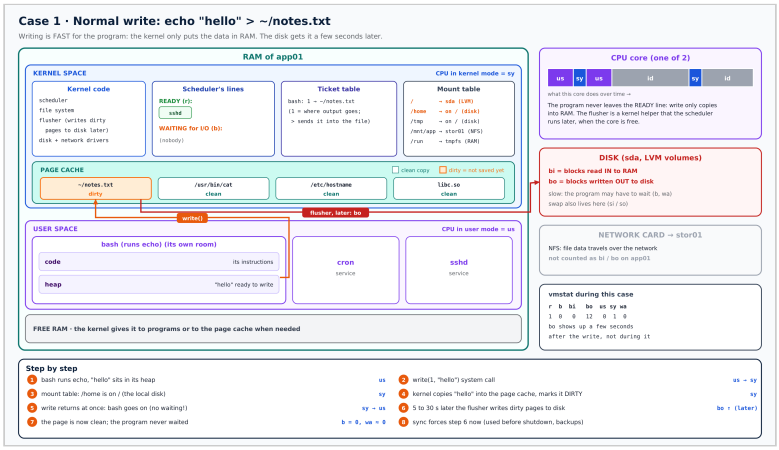
The screenshots explained

dd wrote 300 MB in 0.88 s (359 MB/s) – only into RAM as dirty pages; `sync` wrote them to the disk and `drop_caches` emptied the cache. vmstat shows the quiet “before” picture: skip line 1 (average since boot), then r 0, b 0, bi 0, small bo from background logs, id 97–100, wa 0.

---

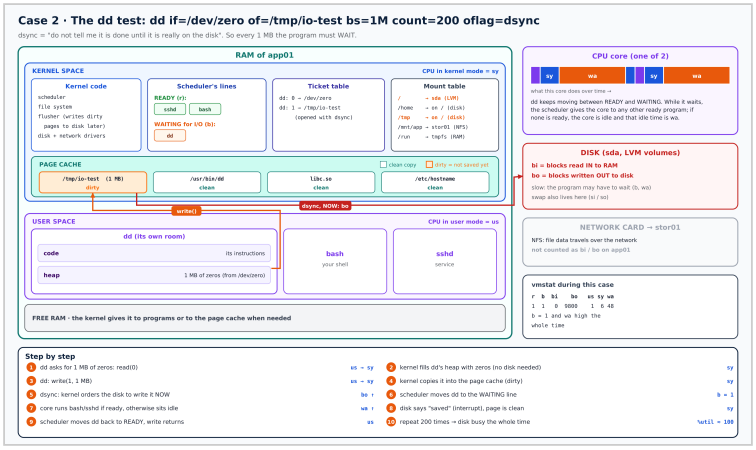
7. Writing, compressing and mounts on the same map

Case 1 – normal write: RAM first (dirty pages), disk later (`bo` later).



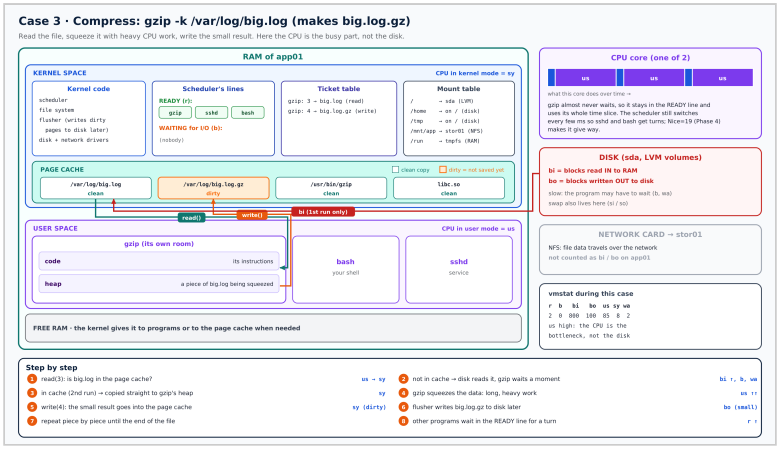
Case 1 – normal write

Case 2 – the dd test: `oflag=dsync` forces every MB to the disk, so dd waits (`b`, `wa`, `bo`).



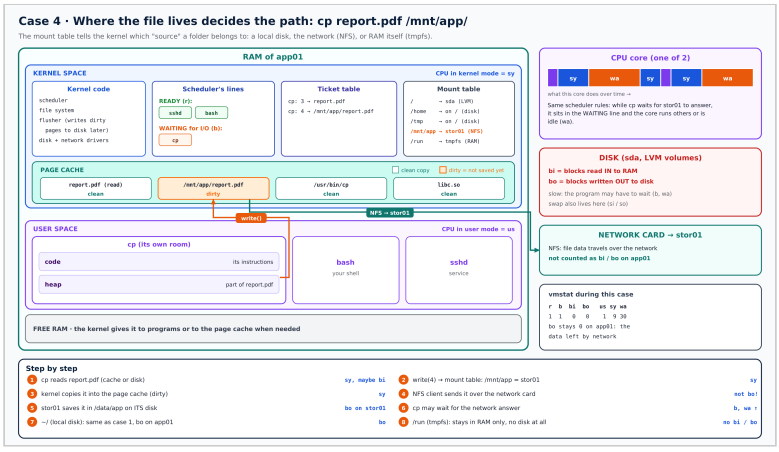
Case 2 – the dd test

Case 3 – compress: the CPU works (us high, r up), the disk is quiet.



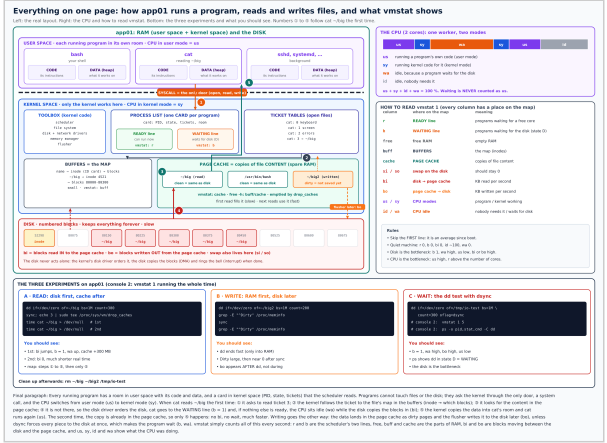
Case 3 – compress

Case 4 – mounts: the folder decides the path – local disk, NFS to stor01 (bo shows on stor01), or tmpfs (RAM only).



Case 4 – mounts

All four cases and the vmstat columns on one page:



Everything on one page

8. Next – Step 5.1 experiments

With `vmstat 1` running in a second console on app01:

- **A · read:** `time cat ~/big > /dev/null` twice → 1st: `bi` up, `b` = 1, `wa` up, cache +300 MB · 2nd: `bi` 0, much faster.
- **B · write:** `dd` without dsync, `grep ^Dirty /proc/meminfo`, `sync` → dirty pages first, `bo` later.
- **C · wait:** `dd … oflag=dsync` with `vmstat 1 5`, `iostat -xz 1 5` and `ps -o pid,stat,cmd -C dd` → `b` = 1, `wa` high, dd in state **D**.

Clean up afterwards: `rm ~/big ~/big2 /tmp/io-test`.

Final understanding (one paragraph)

Every running program has a body in user space (its code and data) and a card in kernel space (PID, parent, state R/S/D, and a pointer to its own file table); bash makes cat's card with the system calls fork and exec, and every request after that also goes through a system call that switches the CPU from user mode (us) to kernel mode (sy). When cat opens F1, the file system first finds the map – the folder entry (name → inode number) and the inode (owner, size, block list) – from the map cache, or from the disk if it is missing, and only then writes ticket 3 → inode in cat's file table. When cat reads 3, the file system follows the inode to the block numbers and takes the content from the page cache, or, if it is missing, the disk driver fetches the content blocks into the page cache while the file system marks cat's card D, the scheduler (which only reads the states) runs other programs or leaves the CPU idle (wa), and the disk driver marks the card R again when the disk rings its bell; finally the content is copied into cat's heap. The second time, everything is already in RAM. Writes go the other way: into the page cache as dirty pages, to the disk later (bo), unless dsync forces them at once and makes the program wait. A file stays alive while any name or any open ticket points to its inode – which is why, in the Phase 4 drill, the deleted log still filled the disk until the process holding it was killed. vmstat simply counts all of this: r and b are the scheduler's lines, free, buff and cache are parts of RAM, si and so are swap, bi and bo are blocks moving between the disk and the caches, and us, sy, id and wa show what the CPU was doing.